

Engineering experience:

1. Describe a skill or knowledge you acquired recently that has been impactful for you. Why did you make this investment? What has the outcome been for you and your team?

In the past few months, I've gained experience with Kubernetes and container orchestration. One of our three DevOps engineers left, so I volunteered to take on basic deployment responsibilities.

I worked on Dockerfiles, Kubernetes manifests (deployments, ConfigMaps, secrets, HPAs), and used Kustomize to manage configurations across Dev and QA environments. I also took ownership of the CI/CD pipelines for the services I'm involved with and learned to write multistage Dockerfiles.

One improvement I designed with my new learning was for one of our custom scanners. Previously, we had to update the entire deployment just to update the scanning templates. I built a GitHub Actions pipeline that pushes templates to our artifact registry, and used a Kubernetes initContainer to pull the latest templates at startup. This segregated template updates from application deployments.

I'm now confident in owning the full deployment lifecycle for my services, and the DevOps team has bandwidth for higher priority complex projects.

2. What new skill would you like to learn? Why do you think this is important or timely or interesting? Why do you think you will be good at it?

I want to develop my skills in AI Engineering. I have enrolled in a certification course by IIT Roorkee, which requires passing an entrance test. I have been experimenting with LangChain in my spare time.

I find AI interesting because it can handle problems where deterministic algorithms fall short. In my work as a developer for a cybersecurity platform, I've experienced this. For example, when analysing leaked credential dumps, we relied on known patterns to extract useful information but there is often more in those files, AI could help surface additional information that generic pattern matching alone would miss.

I'm confident I can learn this well as I am comfortable with Python. I believe I can pick up new technical domains quickly and I have real problems I want to solve, which I find is the best motivation for my learning.

3. What kinds of software projects have you worked on before? Which operating systems, development environments, languages, databases?

I have worked mainly on B2B SaaS products. My previous company built an observability platform for financial data, serving banks and fintech companies. My current company builds an Attack Surface Monitoring and Continuous Automated Red Teaming platform for businesses.

I have been using Ubuntu as my daily driver for the past 10 years. I primarily work in Python, using Django and FastAPI web application frameworks. I've also worked with frontend technologies including React.js and AngularJS. I have worked with relational databases (PostgreSQL & MySQL), implemented a global cache in a distributed environment using Redis, and worked with Elasticsearch as a time series database. I have also worked with Google BigTable.

4. Why Python, and what's your history with this language?

Python is a language that works across many domains, starting from web development, APIs, DevOps scripting, security tooling, automation, and now AI/ML. This suits me well because my interests are broad. I made it a point to get proficient in Python so I could move between different areas without having to learn a new language each time. I also appreciate that Python lowers the barrier for non-programmers like scientists and researchers to build things.

My history with Python started in college. I was stubbornly committed to doing everything in C. During a timed coding interview, the first step of the problem required combining and sorting two input sets.

I wrote a merge sort from scratch, while my friend simply did `(a + b).sort()`.

My logic was correct, but I failed one test case due to a memory allocation bug. That was a turning point for me. I realized that choosing the right tools and abstractions mattered more than proving I could implement everything from first principles, and that Python let me get to correct solutions faster.

I then joined my college's Autonomous Underwater Vehicle team, building an AUV for an international competition in Singapore (SAUVC 2018). I used Python extensively for computer vision systems. We made it to the finals.

Professionally, I've been using Python for 7+ years. I started automating my tasks when I was working at EY as a cybersecurity analyst. They noticed my work and asked me to build a Django application that checked security audit configurations across systems in Active Directory, using Ansible. From there, I moved into web development and have been building Python-based backend systems since.

5. Would you describe yourself as a high quality coder? Why?

I strive to be. According to me, any code has to be maintainable and extensible. I write tests, do code reviews, and document my work. I think about edge cases and avoid shortcuts that create technical debt. When I work with interns, I teach them to write code for extensibility and maintainability, not just code that works today, but code that can evolve over time.

A recent example is when we noticed many of our integrations could be done by security researchers in our company who write scripts, and with AI, the quality of those scripts has improved. So I built a pluggable platform where we can invoke their scripts directly from our platform. Our main goal was to make it extensible and to add new integrations without touching core code. It's actively being used now, with about 15 scripts integrated. In this case, the quality of the platform code directly affects the speed of new integrations.

6. Would you describe yourself as an architect of resilient software? If so, why, and in which sorts of applications?

Yes. I regularly work with asynchronous scan requests which are processed by pods in a distributed environment running on spot nodes. This forces me to design for interruption as pods can go down at any time, so I build systems that checkpoint progress and to resume without interruptions.

A recent example is when I integrated Burp Scanner into our Attack Surface Monitoring Platform. Multiple Burp instances run on separate VMs, and multiple pods with autoscaling handle scan requests. Any pod can pick up a request and assign it to a Burp instance. Since pods run on spot nodes that can be

terminated, I designed a checkpointing system. I regularly fetch partial scan results and save them to cloud storage along with the scan ID and details of which Burp instance handling the scan. If a pod goes down, a new pod can pick up the scan ID, download the checkpoint, and route the request back to the same Burp instance to resume from where it left off, essentially a sticky scan. I designed this particular architecture myself.

I also use retry mechanisms with exponential backoff for network failures, and I have integrated Slack alerts for when scanner instances go down for Customer Success Teams.

7. What software products have you yourself led which shipped many releases to multiple customers? What was your role?

In my current company, I am the primary owner of the integration module of our web application, which has shipped over 80 releases. I have integrated 50+ security tools into our platform. Each integration involves researching the tool, understanding its API or output format, keeping the intermediate results in a knowledge graph, designing database tables to store the results after normalising, writing PostgreSQL queries, and building APIs for the frontend. Maintaining these integrations is an ongoing effort because tools can evolve, deprecate features, or change output formats.

I've been responsible for the full lifecycle, feature development, releases, and production issues.

In my previous company, I owned the authentication, authorization, user management, and access control modules for the main platform. I led the migration from Role-Based Access Control to Claims-Based Access Control, which was a significant architectural change. Additionally, I worked on frontend features using React.js and AngularJS. In my first company, I built a tool that enforced secure configurations across all systems in an Active Directory environment using Ansible.

8. What is your most senior role in a software engineering organisation? Describe your span of control, and the diversity of products, functions and teams you led.

My current role is Senior Engineer. I report to the Director of Engineering, who reports to the CTO.

I don't have direct reports, but I take on leadership responsibilities in other ways. Every six months, we hire 1-2 interns for my team and I conduct interviews, provide knowledge transfer, and enable them to contribute effectively as quickly as possible.

I regularly work cross-functionally by coordinating with frontend engineers on features, working closely with QA during releases, and collaborating with DevOps on deployments. Since I own the integrations module, I frequently work with our Security Researchers to understand their thoughts on improving the product. Recently, I've expanded into DevOps work, deploying my own code to GKE.

9. What is your proudest success as an engineering leader?

In my previous company, I volunteered to be the SPOC for all security audits. Our clients were banks, so security requirements were strict and audits happened for every client. I was hired for a backend role, but when I heard about the audit process, I volunteered to lead it because I had a security background and wanted to help strengthen the product.

I conducted sessions for developers to address the gaps in our codebase. We cut common vulnerabilities by around 90%. When I left, the product was in a much better security posture than when I joined, that's something I'm proud of.

In my current company, I continue this approach by regularly conducting interviews, running knowledge sharing sessions, and voluntarily working with security researchers to bring the best outcomes for the company.

10. Outline your thoughts on open source software development. What is important to get right in open source projects? What open source projects have you worked on? Have you been an open source maintainer, on which projects, and what was your role?

I have not contributed as much to open source as I would like. My contributions so far have been limited to a minor bug fix in Postleaks, an open source security tool. However, I rely heavily on open source in my daily work - Django, Kubernetes, ELK stack, and Ubuntu. I've also had to dig into open source internals when needed, for example, I reverse engineered Elasticsearch to maintain compatibility with older Kibana versions, which required understanding the codebase at a deeper level. The changes were proprietary so I couldn't contribute them upstream.

I believe culture and communication are important in open source projects. Good documentation, clear contribution guidelines, and a low barrier for entry to contribute are essential, especially to encourage younger developers to make their first contributions.

I genuinely love open source software, it was the main reason I switched to Ubuntu in the first place and Firefox as my primary browser. There's something compelling about great minds coming together to build software that anyone can use, learn from, and improve. I would like to contribute more to the community, and joining Canonical would align well with that goal.

11. Describe your experience building large systems with many services - web front ends, REST APIs, data stores, event processing and other kinds of integration between components. What are the key things to think about in regard to architecture, maintainability, and reliability in these large systems?

For the past six years, I've been building systems with multiple services. On the frontend side, I've worked with React.js and AngularJS. For APIs, I use Django and FastAPI. For data stores, I've used PostgreSQL for relational data, Redis for caching, and Elasticsearch for search and time-series data. For event processing, I've worked with messaging queues like Google Pub/Sub. I've also built integrations with systems via REST APIs.

For architecture, I believe in separation of concerns, each component should have a clear responsibility.

I design systems assuming that failures will happen at some point. If something is going to break, I would rather have it break loudly and early than silently corrupting data.

For maintainability, I swear by test driven development and documentation. Code without tests is hard to change confidently, and code without documentation is hard for others to understand.

For reliability, I use patterns like checkpointing and retries with exponential backoff. I also believe observability is essential for logs, metrics, and alerts to help developers understand what exactly is happening in complex systems.

12. How comprehensive would you say your knowledge of a Linux distribution is, from the kernel up? How familiar are you with low-level system architecture, runtimes and Linux distro packaging? How have you gained this knowledge?

I have used Ubuntu as my daily driver for 10 years, so I'm comfortable with Linux from a user and developer perspective. I've written shell scripts to automate tasks, and I've fixed system issues as they've come up, mostly self taught through reading, experimentation and Youtube videos.

However, I'd be honest that my knowledge of low-level system architecture, kernel internals, and Linux distro packaging is limited. I understand concepts at a high level from books and videos, but I haven't had hands-on experience with building packages or working close to the kernel.

This is an area I'd be keen to develop, especially at a company like Canonical where this knowledge is core to the work.

13. Describe your experience with large-scale IT operations, SAAS, or other running services, in a devops or IS or system administration capacity

I have worked on B2B products with different deployment models. In my current company, we run a cloud-based SaaS platform for Continuous Automated Red Teaming. The primary users are the security team at client companies. Even though application user numbers are low, the operational complexity comes from the data side. We scan over a million IPs a day, running daily reconnaissance, OSINT gathering, and vulnerability scans. This means managing multiple scanning services, data pipelines, and ensuring results are fresh and reliable. I am responsible for releases and monitor rollouts, fixing issues as they come up. I've also taken on deployment work which involves managing Kubernetes manifests, CI/CD pipelines, and configuring deployments for Dev and QA environments in GKE.

In my previous company, our software was installed on-premise within the customer's network. I was on call during deployments to address any issues that came up. In my first company, I built a tool that enforced security configurations across systems in an Active Directory environment and also generated compliance reports.

14. Describe your experience with public cloud based operations - how well do you understand large-scale public cloud estate management and developer experience?

I have hands-on experience with Google Cloud Platform, primarily working with GKE (Kubernetes Engine), Pub/Sub, Cloud Storage, and Bigtable. While the DevOps team handles infrastructure provisioning, I work directly with these services for deployments, messaging, and data storage. I've worked with deployments across multiple environments and regions. We noticed that some of our traffic was getting blocked in certain regions, so I implemented a solution that lets users choose which geographical region to run scans in and allows us to overcome regional restrictions. I was also involved in discussions and implementation to reduce cloud operational costs. For non-production environments, we redirected logs from GCP's cloud logging to Cloud Storage instead. Whenever we need to view the data, we push the necessary logs to OpenSearch. This reduced our cloud monitoring costs while keeping logs accessible when needed. I have limited experience with other cloud providers. I have worked with Azure for SSO integration with Azure AD, and used CloudWatch on AWS for log monitoring.

15. Outline your thoughts on quality in software development. What practices are most effective to drive improvements in quality?

Quality in software development means fewer bugs, reliability, and good test coverage. Code should work correctly and continue to work correctly as the codebase evolves.

Practices I have found effective

- 1. Testing: For features that I assign to interns, I write the test cases myself and run them against their code. I believe it's important that someone other than the developer writes the tests. It avoids bias and catches assumptions the developer might have. These tests run during CI checks when building images, ensuring new features don't break existing functionality.*
- 2. Code reviews: I regularly review code and try to give constructive feedback on writing clean, maintainable code.*
- 3. QA process: Our team has a structured QA process, regression testing and manual testing of new features. We push changes to a demo environment first, where our security testers use it for a week before we deploy it to production. This buffer catches issues before they affect real users.*

16. Outline your thoughts on documentation in large software projects. What practices should teams follow? What are great examples of open source docs?

Documentation is essential in large software projects. The key to good documentation is knowing your audience and keeping it maintained. Different audiences need different documentation.

Developers need API references and code examples. End users need guides written in plain language. New team members need onboarding docs. I believe end user documentation especially benefits from having a separate team handle it. In my experience, developers often write in a way that only other developers understand, which doesn't serve non-technical and new users well.

Documentation should be treated as part of the codebase, reviewed like code is reviewed, and updated when code changes. A controversial opinion I hold is that for developers and people willing to get their hands dirty, outdated documentation is worse than no documentation at all. With no docs, you ask someone or dig into the code. With outdated docs, you trust them and waste time before questioning them.

For great examples of open source documentation:

- 1. Django - I like how they have separate sections for tutorials and references. You don't have to leave their documentation site to get something done. It works for beginners following tutorials and experienced developers looking up specific APIs.*
- 2. Kubernetes - Since it's a concept-heavy tool, I appreciate how they explain all the concepts first before getting into implementation. They also provide basic manifests that actually work for the majority of cases, so you can get started quickly and then customize.*

17. Outline your thoughts on user experience, usability and design in software. How do you lead teams to deliver outstanding user experience?

I have worked mostly on B2B SaaS products for technical users who are security researchers and enterprise teams.

These are the things I've contributed to for better user experience:

- 1. Frontend Performance: I minified JavaScript in our SPA so the home page loads faster, especially for users on slow networks.*
- 2. Safe: Users should be able to explore without risk of breaking things. When I worked on user management, I removed the admin user from the list of visible users and roles. Normal users don't need to see it, and hiding it prevents accidental edits or deletion of critical permissions.*
- 3. Smart defaults: In our platform, CXO users and hands-on security researchers have different needs. I made home pages configurable by user type in the user management settings, so each user sees what's relevant to them.*
- 4. Familiarity: When building features, I try to map the flow to what users are already familiar with. If they use another tool, it doesn't hurt to mimic that flow rather than making them relearn.*

18. Outline your thoughts on performance in software engineering. How do you ensure that your product is fast?

According to me, performance and its priority in software engineering changes based on the type of application, the stage the product is in, the stage the company is in, and the users of the product.

I've always considered performance optimization as an important backlog to address when necessary, with exceptions for a few low-hanging fruits like using async calls for REST API integrations, it's an easy decision that can significantly improve response times. I believe it's better to have a working product first and then optimize, if needed. I often find users more forgiving of some slowness when the correct UX techniques are used.

Examples where I have prioritized performance:

- 1. DB Query Optimization: A data-heavy API was taking 60 seconds to respond. I added indexes, changed the join order, and used jsonb_agg only at the end for the selected paginated data instead of aggregating everything upfront. This brought it down to 6 seconds.*
- 2. Global Caching: Implemented Redis caching in a distributed environment with invalidation strategies to maintain data freshness.*
- 3. Custom HPA: Built a custom Horizontal Pod Autoscaler for scanner pods based on the number of scan request messages in Google Pub/Sub.*
- 4. Scanner Pod Efficiency: Our scanner pods have multiple processes subscribing to the same topic, plus an abort listener process that maps scan IDs to their processes. When a scan is aborted, the abort process identifies which process is handling it and kills it, freeing up resources for other scans instead of wasting compute on aborted work.*
- 5. Denormalization: Made deliberate decisions to denormalize data and restructure tables to reduce joins where read performance was critical, trading speed over storage.*

19. Outline your thoughts on security in software engineering. How do you lead your engineers to improve their security posture and awareness?

I view security in software engineering as an ethical obligation to the users and to the company, rather than just a technical requirement. My career started in security, and that shaped my core philosophy: 'Do no harm.' A security vulnerability can be damaging to the users and the company. Because of that, security can never be treated as technical debt, it has to be baked into the product from day one.

1. *How I lead my teams*

I've found that developers don't need more checklists, they need a change in perspective. I use my Red Team background to run sessions where we look at our own code from an attacker's point of view. For example, I often share a 'war story' from my early days where I bypassed the payment gates of a platform like Audible. By simply mimicking a '200 OK' response in the mobile application, I could download books for free because they relied entirely on client-side validation. When developers see how a simple logical oversight results in direct financial loss, it changes how they write code forever. They stop trusting the client.

2. *Architectural Approach*

Technically, I insist on intentional security. Whenever I create an architecture, I separate the security components into distinct layers. This distinction is critical. A failure in business logic is a bug. A failure in the validation or authorization layers is a liability. By creating these deliberate chokepoints, we ensure that external input is scrubbed and permissions are verified before they ever touch the business logic. It makes the system much harder to bypass.

3. *Security in Open Source*

Finally, regarding Open Source, I advocate for a 'Trust, but Verify' model. The XZ Utils backdoor taught us that even long-term contributors can be a risk. While OSS is a strength because anyone can find and fix bugs, it also requires us to be disciplined. In my teams, that means mandatory dependency scanning and CI/CD audits are standard, ensuring we benefit from the transparency of open source without inheriting its hidden risks.

20. Outline your thoughts on devops and devsecops. Which practices are effective, and which are overrated?

To me, DevOps is about creating the fastest reliable path to production through a systematic approach. The most effective practices start with developers, from version control to tests and this is where DevOps supports the entire process.

DevSecOps builds on top of this by treating security as part of the system itself. Secure infrastructure as code, automated secrets management, and policy enforcement should be default behaviors. I strongly believe in shift left security, catching authorization and logic flaws early.

I'm also cautious of metrics and controls that look good on paper but fail in practice. Chasing 100% test coverage often hides weak test quality, and heavy manual approval gates usually signal a lack of trust in the automation that's meant to protect the system.

Industry leadership experience

21. Describe your speaking experience at industry events and conferences

I haven't spoken at industry conferences, but I have delivered security awareness training to non-technical audiences. While working at my first company, a steel plant lost a significant amount of money in a phishing scam and needed someone who could conduct an awareness session in Tamil. I conducted four one-hour sessions for different groups. Workers and management, tailoring the content for each audience. We tested their understanding before and after the training.

22. Are you a thought leader in any particular area of technology?

I can see myself as a thought leader in the future, particularly at the intersection of domains. One area I'm passionate about is secure software development. It's often assumed that developers don't prioritize security, and I want to break that mindset. While I'm not a thought leader yet, my goal is to promote best practices, raise awareness, and eventually establish myself in this space.

23. Describe any experience working with startups. What did you draw from that experience that would be relevant for this application?

I've worked at two startups. The biggest lesson has been flexibility and ownership, you can never say "it's not my job" and walk away. When something needs to be done, you step in. As I mentioned in my first answer, when our DevOps engineer left, I volunteered to take on deployment responsibilities to keep the team moving. Startups have also taught me the importance of small and fast release cycles, fail fast strategies, and the ability to pivot quickly when something isn't working. This translates well to Canonical's environment. Working across the open source environment requires adaptability and willingness to contribute beyond a narrow role.

24. Describe any experience working in a public company. What is important for your colleagues to know about being a public company?

I haven't worked for a public company. My experience has been with startups and a private company.

Education

25. How did you rank in your final year of high school in mathematics? Were you a top student? On what basis would you say that?

I scored 198 out of 200 in mathematics in my final year of high school. Only about 5 students out of a batch of 600 scored higher, placing me in the top 1% of my class.

26. How did you rank in your final year of high school, in your home language? Were you a top student? On what basis would you say that?

I scored 190 out of 200 in English in my final year of high school, which was still among the top scorers in my batch.

27. Please state your high school graduation results or university entrance results, and explain the grading system used. For example, in the US, you might give your SAT or ACT scores. In Germany, you might give your scores out of a grading system of 1-5, with 1 being the best.

I completed my high school under the Tamil Nadu State Board, scoring 1170 out of 1200 (97.5%). The grading system is marks based out of 1200 total across six subjects.

28. Can you make a case that you are in the top 5% in your academic year, or top 1%, or even higher? If so please outline that case. Make reference where possible to standardised testing results at regional or national level, or university entrance results. Please explain any specific grading system used.

*In my high school board exams (Tamil Nadu State Board), I scored 1170/1200 (97.5%)
I was the only student in my batch of 600 to pass JEE Mains as a General Category student while studying under State Board curriculum, which is notable since the Tamil Nadu State Board syllabus doesn't align closely with JEE compared to CBSE*

29. What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for?

I was academically focused but also had a strong interest in general knowledge and quizzing. I participated in quiz competitions regularly and had some success. I won a quiz competition on radio in Chennai and a state-level quiz in Tamil Nadu that was televised on Podhigai TV channel. My high school peers would probably remember me as a nerd.

30. Which university and degree did you choose? What other universities did you consider, and why did you select that one?

I studied Computer Science and Engineering at Sri Venkateswara College of Engineering, affiliated with Anna University. I was selected for NIT but did not get Computer Science as a branch. I was also selected for VIT but could not afford the fees. I chose SVCE because it offered Computer Science at an affordable cost. For me, studying the subject I was passionate about mattered more than the institution's brand name.

31. Overall, what was your degree result and how did that reflect on your ability? Please help us understand the grading system for your results.

I graduated with a CGPA of 8.42 out of 10. Anna University uses a 10-point CGPA system, where 8.5 and above is considered first class with distinction. I was among the top 10 students in my batch of 300.

32. During all of your education years, from high school to university, can you describe any achievements that were truly exceptional?

I won a hackathon where I was funded to develop an Android application that teaches the basics of Biotech through puzzles (simulating recombinant DNA technology, cutting DNA with restriction enzymes and joining them with ligase) and quizzes. The app was published on the Google Play Store for about a year and was used to represent our college at the iGEM event, where the team won a silver medal.

I won the Student Research Award in my department for developing an application that could take a picture of a Sudoku puzzle and solve it using image recognition.

I was part of the team that reached the finals at the Singapore Autonomous Underwater Vehicle Challenge (SAUVC 2018), an international competition.

33. What leadership roles did you take on during your education? Did you conceive of, and drive to completion, any initiatives outside of your required classwork?

Leadership roles:

- *President of Rotaract Club - Led volunteer activities including tree planting drives and providing materials to college gardeners.*
- *Co-founded the Quizzers Forum - Organised quizzes for all events in our college.*
- *Member of Association of Computer Machinery and ACE, taking on a middle management role in my final year.*

Initiatives outside coursework:

- *Conceived and drove the Biotech app from hackathon win to Play Store publication.*
- *Built the Sudoku solver independently as a personal project.*
- *Contributed to the AUV team's computer vision systems for the international competition.*

34. Outline your thoughts on the mission of Canonical. What is it about the company's purpose and goals which is most appealing to you? What do you see as risky or unappealing?

Canonical's mission is to make open source software more secure, reliable, and accessible at global scale. Ubuntu is a clear example of how community driven software can also meet enterprise expectations.

What I find most appealing is Canonical's focus on foundational infrastructure. Operating systems, packaging, Kubernetes, automation, and lifecycle management.

The emphasis on openness, both in licensing and in technical direction is also important to me. Canonical operates in a way that respects user freedom while still engaging with enterprise requirements, which is a difficult but valuable balance.

35. Who are Canonical's key competitors, and how should Canonical set about winning?

I believe Canonical's main competitors are Red Hat (now IBM), SUSE, the major cloud providers (AWS, Azure, GCP), VMware, and HashiCorp.

Red Hat has enterprise credibility and a massive sales force. Cloud providers have lock in, once someone is on AWS, it's easier to stay on AWS. VMware owns the virtualization layer in most enterprises. HashiCorp has become the default for infrastructure automation.

Canonical's advantage is different, it's cloud-agnostic and open. Ubuntu runs everywhere, on any cloud, on bare metal, on IoT devices. The stack is integrated, from OS to containers to Kubernetes to lifecycle management, without tying anyone to one vendor.

To win, I think Canonical needs to tell its story to a wider audience. The products are strong, but too many people don't know what Canonical does beyond Ubuntu.

36. Why do you most want to work for Canonical?

What first drew me to Ubuntu was its meaning, "I am because we are." That philosophy resonated with me, and I believe it captures the essence of open source. We build on each other's work, we share, we improve together. I have been using Ubuntu ever since my college days.

I want to work for Canonical because it operates at the intersection of open source values, real world scale, and engineering discipline. The company works on problems that matter. In my own experience, open source has been both a learning platform and a collaboration model. Throughout my career, I've relied on many open source tools that were critical to the success of my projects, I believe Canonical stands at the forefront of this motto.

37. What would you most want to change about Canonical?

Canonical has built many successful products beyond Ubuntu like Snaps, OpenStack and much more, along with efforts like Ubuntu Phone that didn't take off. But outside of Ubuntu itself, I personally feel these aren't widely known. For example, many people around me had never heard of Ubuntu Phone, or even know that Canonical has an entire ecosystem of tools for containers, Kubernetes, and bare metal provisioning. These products solve real problems, but the story isn't reaching the broader community.

If I could change one thing, it would be improving how Canonical's work is documented, explained, and shared, so that strong engineering efforts translate into wider awareness and adoption. That's something I'd want to help with.

38. What gets you most excited about this role?

I've used Ubuntu as my daily driver for 10 years. The idea of contributing to something I've relied on every day is exciting in itself. But what makes this role a genuine fit is the intersection with my background, which is high quality Python code with focus on security.

I'm also excited to collaborate with people around the world and also, I love debugging. There's something satisfying about tracing a problem to its root. I'd be happy doing that for software that matters.